Vite & Gourmand

Documentation technique

Choix techniques, environnement, modèle de données, diagrammes, sécurité et déploiement

---

ECF – Titre professionnel Développeur Web et Web Mobile (Studi)

Auteur : Quentin Cire

Site : https://vite-gourmand-kappa.vercel.app

Dépôt : https://github.com/Bossim-Rack/V-G

Version du 29 octobre 2026

1. Présentation

Vite & Gourmand est un traiteur bordelais tenu par Julie et José. L'application permet de **présenter les menus à tous les visiteurs** et de **les commander en ligne**. L'équipe gère le catalogue, les commandes et les avis, et l'administrateur suit l'activité grâce aux statistiques.

L'application est découpée en deux parties indépendantes qui communiquent en JSON :

- un **front** en HTML, CSS et JavaScript, hébergé sur Vercel ;
- une **API REST** en PHP avec Symfony, hébergée sur Render, reliée à une base **MySQL** (données métier) et à une base **MongoDB** (statistiques).

Élément	Adresse
Site	https://vite-gourmand-kappa.vercel.app
API	https://v-g.onrender.com/api
Documentation interactive de l'API	https://v-g.onrender.com/api/doc
Code source	https://github.com/Bossim-Rack/V-G

2. Réflexions initiales sur les technologies

Le sujet n'impose qu'une chose : utiliser une base relationnelle **et** une base non relationnelle. Les autres choix ont été faits selon trois critères : ce que le projet demande vraiment, ce que je maîtrise ou peux apprendre dans le temps de l'ECF, et ce qui peut être hébergé gratuitement.

Front : HTML, CSS, Bootstrap et JavaScript sans framework

Option	Avantages	Inconvénients	Décision
JavaScript sans framework	Aucun outil de compilation, maîtrise complète du DOM et de `fetch`, cohérent avec la pile proposée par le sujet	Plus de code à écrire pour le routage et l'affichage	**Retenu**
React ou Vue	Composants, état réactif, écosystème	Outillage (build, bundler) et courbe d'apprentissage pour un site de taille modeste	Écarté

- **Bootstrap 5.3** fournit la grille responsive (le site doit être utilisable sur mobile) et des composants accessibles prêts à l'emploi : menu repliable, fenêtres modales, formulaires.
- Un **petit routeur maison** (`rooter.js`) donne au site le confort d'une application à page unique. Chaque page est un fragment HTML chargé avec `fetch`, l'adresse change avec l'API History (`pushState`), et le bouton « retour » du navigateur fonctionne.
- **Chart.js** dessine le graphique des commandes par menu de l'espace administrateur.

Back : Symfony plutôt que PHP natif

Le sujet propose « PHP avec PDO ». J'ai choisi le framework **Symfony 5.4** (version LTS, maintenue pour la sécurité jusqu'en février 2029), car il apporte directement des briques que j'aurais dû écrire et sécuriser moi-même :

Besoin	Ce que fournit Symfony
Accès à MySQL sans injection SQL	**Doctrine ORM** : requêtes préparées, entités, migrations versionnées
Mots de passe	Composant **PasswordHasher** (bcrypt, algorithme `auto`)
Authentification et droits	Composant **Security** : authentificateur par jeton, `access_control` par rôle, hiérarchie des rôles
Validation des saisies	Composant **Validator** (contraintes sur les entités, groupes de validation)
Mails	**Mailer** + **Twig** pour les gabarits HTML
Documentation de l'API	**NelmioApiDocBundle** (`/api/doc`)
Appels depuis le front sur un autre domaine	**NelmioCorsBundle** (CORS limité à l'adresse du front)

Doctrine utilise PDO en interne : le SQL reste maîtrisé, comme le montrent les fichiers `database/01_creation_base.sql` et `02_donnees_demo.sql`, écrits à la main et identiques au schéma des entités.

API REST et authentification par jeton

Le front et l'API sont sur deux domaines différents (Vercel et Render). J'ai donc choisi une **API sans état** : à la connexion, l'API renvoie un **jeton** (`apiToken`) que le front envoie ensuite dans l'en-tête `X-AUTH-TOKEN`. Ce choix évite les cookies de session entre domaines et les attaques CSRF qui vont avec. Sa contrepartie est que le jeton est stocké dans le `localStorage`, donc lisible par un script injecté. C'est pourquoi tout texte saisi par un utilisateur est échappé avant affichage (voir la partie Sécurité).

Bases de données

- **MySQL 8** pour les données métier (comptes, menus, plats, commandes, avis) : les données sont fortement liées entre elles, et les clés étrangères garantissent leur cohérence. MySQL est la base que je connais le mieux, et Aiven en propose une offre gratuite.
- **MongoDB** pour les statistiques, comme le demande le sujet. Chaque commande y est copiée sous forme de document « à plat » (menu, montants, statut, dates). Le nombre de commandes et le chiffre d'affaires par menu sont calculés par des **pipelines d'agrégation** (`$match`, `$group`, `$sort`), sans jointure. L'API utilise directement le pilote de l'extension PHP `mongodb`, sans bibliothèque supplémentaire.

Hébergement

Partie	Hébergeur	Pourquoi
Front	**Vercel**	Hébergement de fichiers statiques gratuit, HTTPS automatique, déploiement à chaque push
API	**Render** (conteneur Docker)	Offre gratuite capable d'exécuter PHP ; le `Dockerfile` décrit exactement l'environnement (PHP 8.3, Apache, extensions)
MySQL	**Aiven**	MySQL 8 gratuit avec connexion chiffrée (TLS)
MongoDB	**MongoDB Atlas** (cluster M0, Paris)	Offre gratuite officielle
Mails	**Brevo** (API HTTP)	Render bloque le SMTP sur l'offre gratuite ; Brevo accepte l'envoi par une simple requête HTTPS

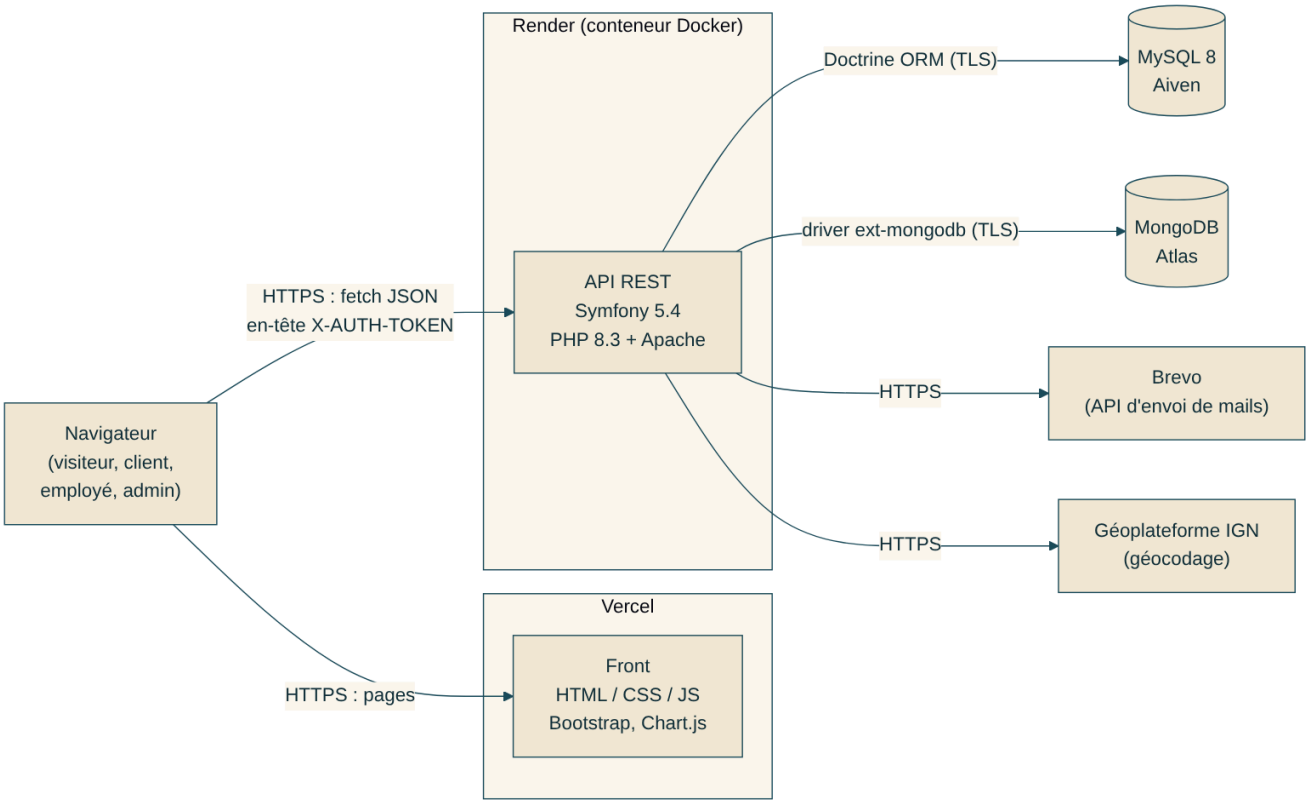
3. Environnement de travail

Outil	Usage
macOS	Système de développement
Visual Studio Code	Éditeur de code (PHP, JavaScript, HTML, CSS)
PHP 8, Composer, Symfony CLI	Exécution de l'API et gestion de ses dépendances
MySQL 8 + MySQL Workbench	Base locale ; Workbench sert aussi à se connecter à la base Aiven en production
Live Server (port 5500)	Serveur local du front pendant le développement
Git et GitHub	Versions du code, branches `main`, `develop` et `feature/...`
draw.io	Premières versions du MCD (dossier `Suivie Projet/`)
`/api/doc` (NelmioApiDoc)	Tester les routes de l'API depuis le navigateur
PHPUnit	Tests automatisés de l'API (`php bin/phpunit`)

Configuration locale :

- Les variables propres à chaque machine (accès MySQL, MongoDB, mails) sont dans `Back-End/synfony V&G/.env.local`, qui n'est jamais versionné.
- Le fichier `.env` versionné ne contient que des valeurs sans danger pour le développement.
- Côté front, `config.js` choisit automatiquement l'API locale (`http://127.0.0.1:8000/api`) quand le site est ouvert sur `localhost`, et l'API en ligne sinon.
- La procédure d'installation complète se trouve dans le `README.md`.

4. Architecture



Le navigateur charge le front sur Vercel puis appelle l'API sur Render, qui seule accède aux bases de données et aux services externes.

Organisation du code

Front (`Front-end/`)

- `index.html` : page unique qui charge Bootstrap, les polices, `config.js` et le routeur.
- `assets/JS/rooter.js` et `Allroutes.js` : table des routes (adresse, titre, fragment HTML, script de la page).
- `assets/pages/*.html` : contenu de chaque page ; `assets/JS/*.js` : le code de chaque page (`menus.js`, `profil.js`, `tableaubord.js`…).
- `Component/Header.html` et `Footer.html` : en-tête et pied de page communs. Le menu s'adapte au rôle stocké à la connexion.

API (`Back-End/synfony V&G/src/`)

Dossier	Contenu
`Controller/`	Une classe par ressource : `MenuController`, `CommandeController`, `AvisController`, `SecurityController` (comptes), `StatsController`…
`Entity/` et `Repository/`	Les tables MySQL et leurs requêtes (ex. `CommandeRepository::rechercher()` pour le filtre des commandes)
`Service/`	`TarifCommandeService` (prix, réduction, livraison), `MailerService` (mails), `StatsMongoService` (MongoDB)

Dossier	Contenu
`Security/`	`ApiTokenAuthenticator` (lecture du jeton) et `UserChecker` (compte désactivé, email non confirmé)
`Command/`	`app:init-database` (base vide prête à l'emploi) et `app:stats:sync` (copie MySQL vers MongoDB)
`Mailer/`	Transport d'envoi par l'API HTTP de Brevo

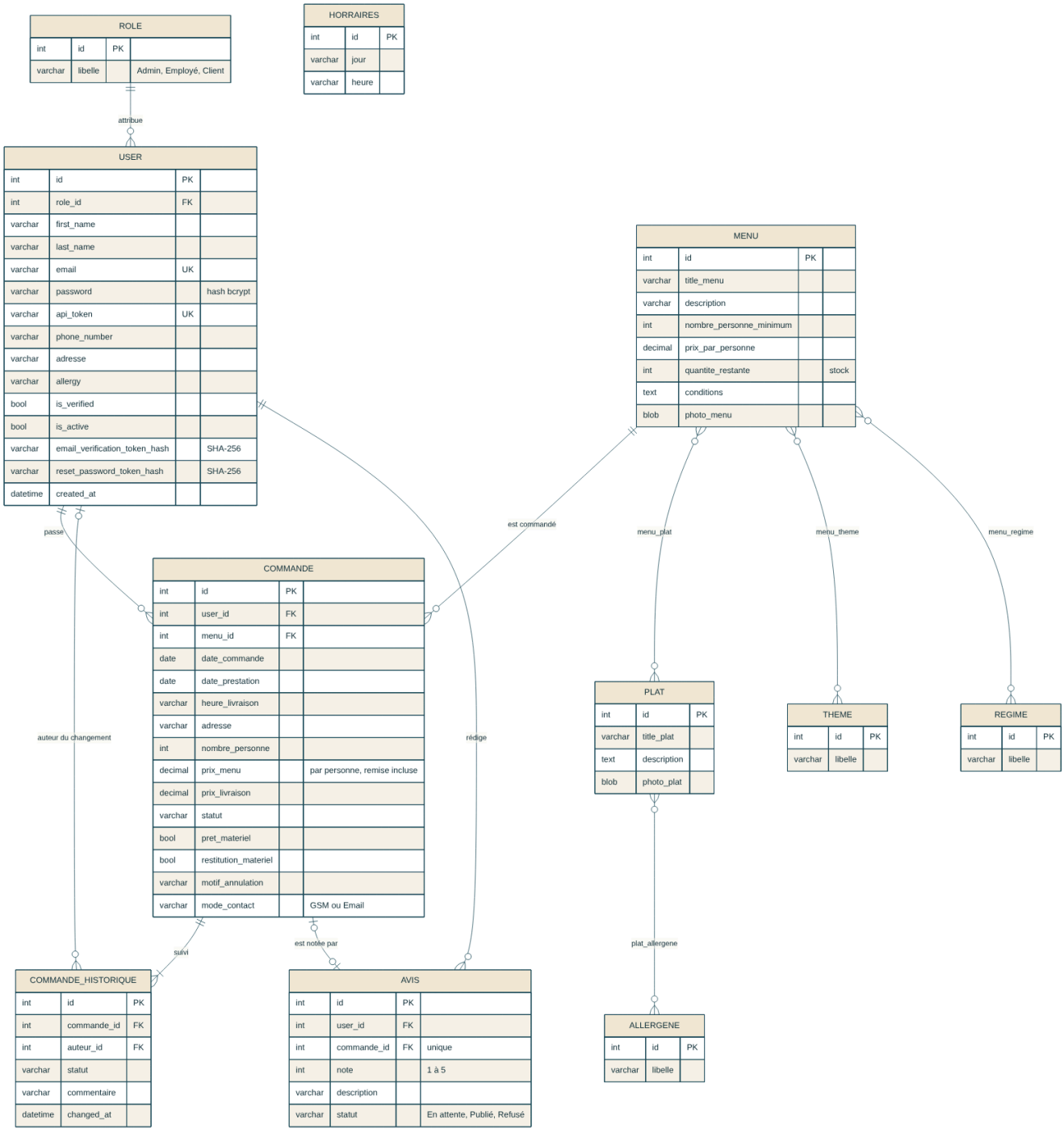
Règles métier côté serveur

Toutes les règles du sujet sont appliquées par l'API. Le front ne fait qu'afficher les résultats et ne peut pas les contourner :

- **Prix** : prix par personne × nombre de personnes, **-10 %** dès 5 personnes de plus que le minimum du menu. Le prix est toujours recalculé par l'API ; un prix envoyé par le navigateur est ignoré.
- **Livraison** : gratuite à Bordeaux, sinon **5 € + 0,59 € par kilomètre**. L'adresse est géocodée par le service de la Géoplateforme (IGN), puis la distance est calculée à vol d'oiseau depuis le centre de Bordeaux (formule de haversine).
- **Stock** : chaque commande diminue le stock du menu, une annulation le rétablit, et une commande est refusée si le stock est épuisé.
- **Cycle de vie d'une commande** : En attente → Accepté → En préparation → En cours de livraison → Livré → (En attente du retour de matériel) → Terminée, ou Annulée. Chaque changement est enregistré avec sa date, son heure et son auteur. Une commande terminée ou annulée ne peut plus changer.
- **Modification par le client** : possible uniquement tant que la commande est « En attente », sans pouvoir changer de menu.
- **Modification par l'équipe** : l'employé doit indiquer le mode de contact du client (GSM ou Email) et un motif.
- **Avis** : un seul par commande, seulement si elle est terminée ; il n'est visible sur l'accueil qu'une fois validé par un employé.

5. Modèle de données

Base relationnelle (MySQL)



Modèle de données : 11 entités et 4 tables de liaison pour les relations plusieurs à plusieurs.

Points de conception :

- **Plusieurs à plusieurs** : un plat peut appartenir à plusieurs menus, un menu à plusieurs thèmes et régimes, un plat peut contenir plusieurs allergènes. Ces relations passent par les tables `menu_plat`, `menu_theme`, `menu_regime` et `plat_allergene`, avec suppression en cascade des liaisons.

- **Prix figés** : `commande.prix_menu` (prix par personne, remise incluse) et `prix_livraison` sont copiés au moment de la commande. Changer le prix d'un menu ne modifie donc pas les commandes passées.
- **Suivi** : `commande_historique` garde une ligne par changement de statut. Si l'auteur est supprimé, `auteur_id` passe à `NULL`, mais l'historique est conservé.
- **Avis** : `avis.commande_id` est unique, ce qui garantit un seul avis par commande.
- **Comptes** : l'email est unique et le rôle obligatoire. Les jetons envoyés par mail ne sont stockés que sous forme d'**empreinte SHA-256**, avec une date d'expiration.

Le script `database/01_creation_base.sql` crée ces 15 tables, et `database/02_donnees_demo.sql` les remplit avec des données de démonstration. Après import, `doctrine:schema:validate` confirme que la base est identique aux entités.

Base NoSQL (MongoDB)

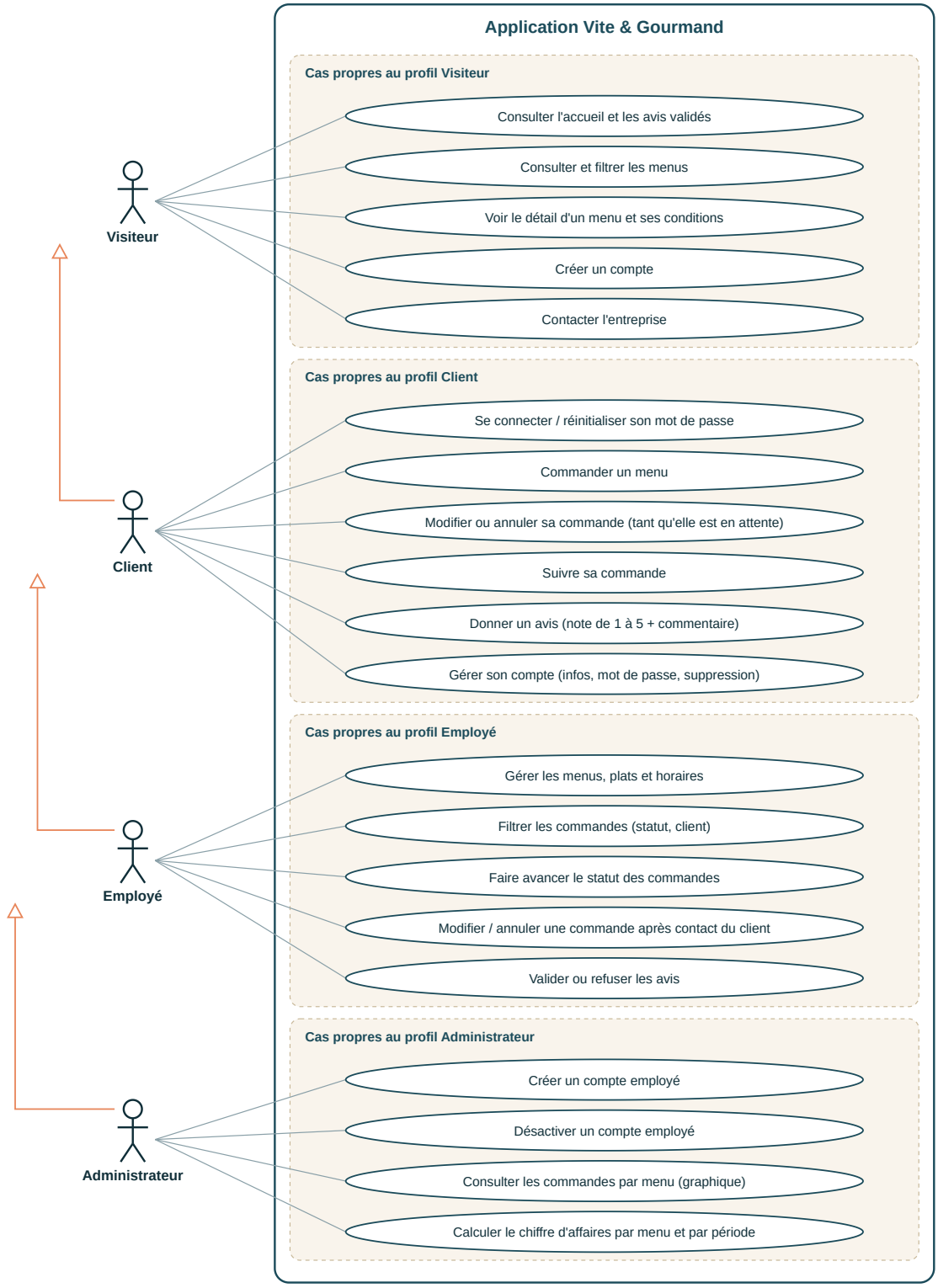
Base `vite_gourmand`, collection `commandes`, avec un document par commande (même identifiant que dans MySQL) :

```
{
  "_id": 3,
  "menuId": 6, "menuTitre": "Menu Cocktail Événement",
  "userId": 5,
  "nombrePersonne": 25,
  "montantMenu": 540.0, "montantLivraison": 8.36, "montantTotal": 548.36,
  "statut": "En attente du retour de matériel",
  "dateCommande": ISODate("2026-09-12"), "datePrestation": "2026-10-03"
}
```

- Le document est **créé ou mis à jour** (upsert) à chaque création, modification, annulation ou changement de statut d'une commande.
- La commande `app:stats:sync`, lancée à chaque démarrage de l'API, reconstruit toute la collection depuis MySQL. Une panne temporaire de MongoDB est donc rattrapée automatiquement.
- Le document ne contient **aucune donnée personnelle**, seulement l'identifiant du client.
- Le chiffre d'affaires additionne `montantTotal` et exclut les commandes annulées. Il peut être filtré par menu et par période (`$match` sur `menuId` et `dateCommande`).

6. Diagrammes

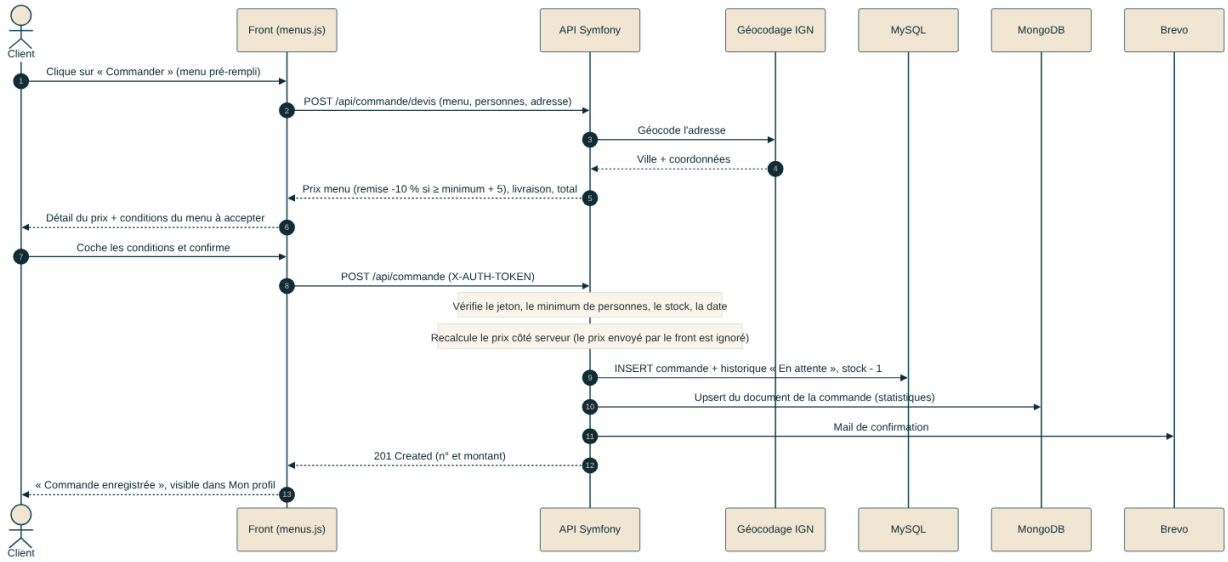
Diagramme de cas d'utilisation



Flèche orange à triangle creux : héritage (chaque acteur peut aussi faire tout ce que fait l'acteur pointé).

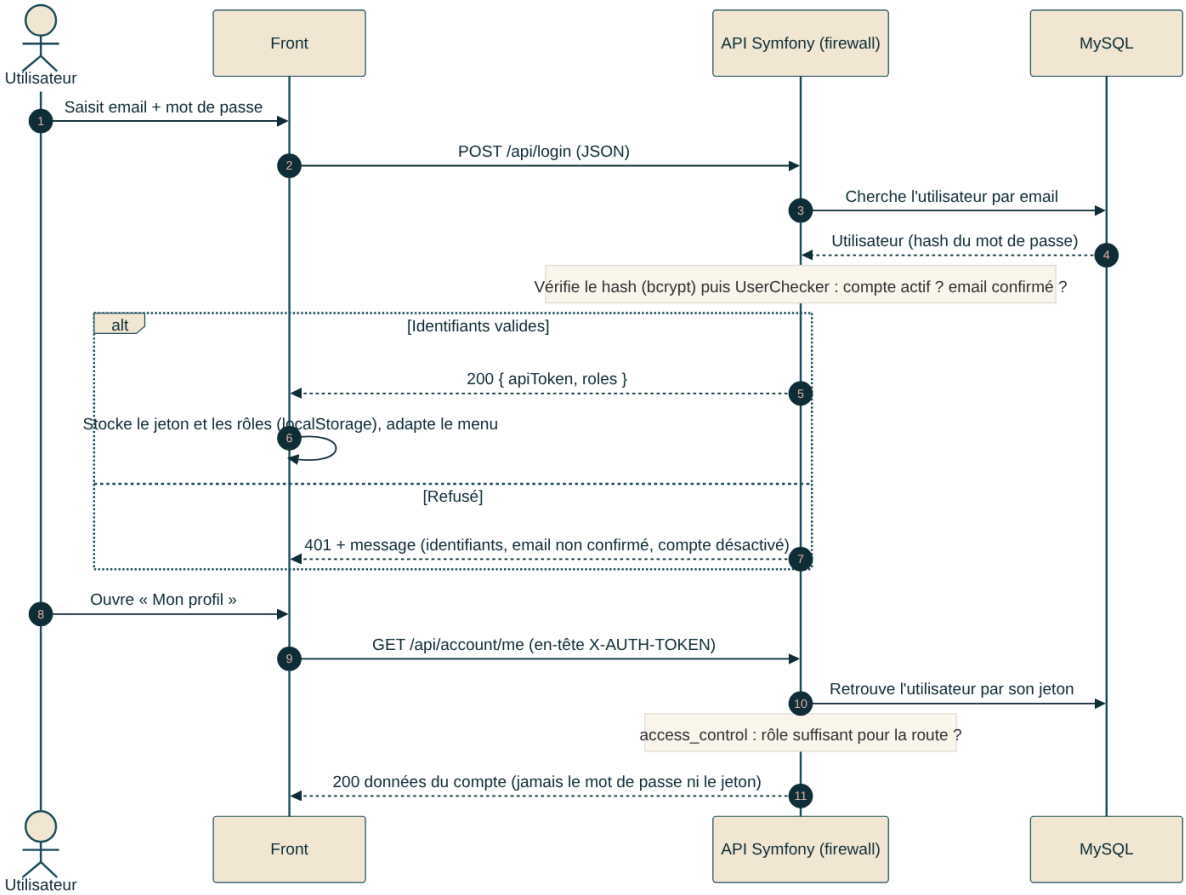
Chaque profil hérite des possibilités du profil au-dessus : le client fait tout ce que fait un visiteur, l'administrateur tout ce que fait un employé.

Diagramme de séquence : passer une commande



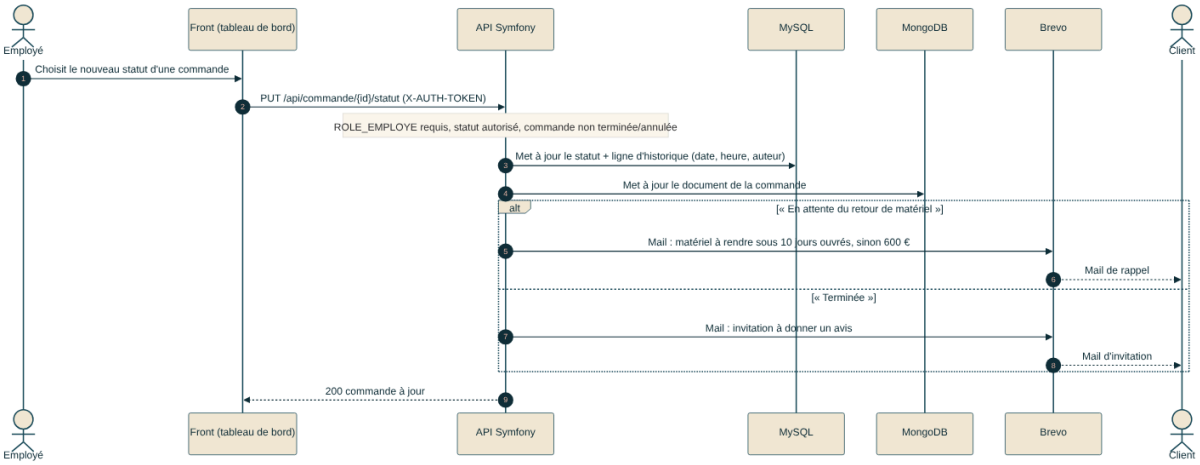
Le prix affiché avant validation vient de l'API (devis). À la validation, l'API recalcule tout avant d'enregistrer.

Diagramme de séquence : connexion et appel authentifié



Le mot de passe n'est vérifié qu'une fois ; les appels suivants utilisent le jeton.

Diagramme de séquence : changement de statut et mails automatiques



Les mails de rappel du matériel et d'invitation à donner un avis partent automatiquement selon le nouveau statut.

7. Sécurité

Comptes et authentification

- **Mots de passe** hashés avec l'algorithme `auto` de Symfony (bcrypt, coût 13), jamais stockés ni renvoyés en clair.
- **Politique de mot de passe** du sujet : 10 caractères minimum, avec majuscule, minuscule, chiffre et caractère spécial. Elle est vérifiée côté front pour l'aide à la saisie, et **côté API** (contraintes du Validator sur le mot de passe en clair, avant hashage).
- **Confirmation de l'email** obligatoire avant la première connexion. Le lien envoyé par mail est valable 48 heures.
- **Mot de passe oublié** : lien à usage unique valable 1 heure. La réponse est la même que l'email existe ou non, ce qui empêche de deviner les comptes inscrits.
- **Jetons des liens** : générés avec `random_bytes(32)` ; la base n'en garde que l'empreinte SHA-256. Une fuite de la base ne permet donc pas de réutiliser un lien.
- **Jeton d'API régénéré** lors d'un changement de mot de passe, d'une désactivation ou d'une suppression de compte : les sessions ouvertes sont coupées.
- **Comptes désactivés** : le `UserChecker` refuse leur connexion, et leur jeton n'est plus valable.

Droits d'accès

- **Rôles** `ROLE_USER` (client), `ROLE_EMPLOYE` et `ROLE_ADMIN`. L'administrateur hérite des droits de l'employé (`role_hierarchy`).
- `access_control` dans `security.yaml` : seules les lectures publiques (menus, thèmes, régimes, allergènes, horaires, avis publiés) et les routes de compte (inscription, connexion, liens par mail, contact) sont ouvertes. La gestion du catalogue exige `ROLE_EMPLOYE`. Les comptes et les statistiques exigent `ROLE_ADMIN`. Tout le reste exige d'être connecté.
- **Propriété des données** : un client ne peut lire, modifier ou annuler que **ses** commandes. Pour une commande d'un autre client, l'API répond « introuvable » (404), sans même confirmer qu'elle existe.
- **Aucun compte administrateur** ne peut être créé depuis l'application. L'administrateur ne peut créer que des employés, et ne peut attribuer que les rôles Client et Employé.

Données et échanges

- **Injection SQL** : toutes les requêtes passent par Doctrine (requêtes préparées).
- **XSS** : tout texte venant d'un utilisateur (noms, adresses, avis, messages) est échappé par la fonction `echapperHtml()` avant d'être inséré dans la page.
- **Validation** de toutes les saisies côté API : formats, longueurs, valeurs autorisées (statuts, modes de contact), dates, nombre minimum de personnes.
- **CORS** : l'API n'accepte les appels que depuis l'adresse du front (variable `CORS_ALLOW_ORIGIN`).
- **Chiffrement** : HTTPS partout (Vercel et Render), connexion TLS à MySQL (certificat CA d'Aiven) et à MongoDB Atlas.

- **Secrets** : clés et mots de passe uniquement dans les variables d'environnement de Render, jamais dans le dépôt.
- **Formulaire de contact** protégé contre les robots par un champ piège invisible (« honeypot »).

RGPD

- **Minimisation** : seules les données nécessaires à la commande et à la livraison sont collectées. MongoDB ne contient aucune donnée personnelle.
- **Droit à l'effacement** : le client peut supprimer son compte depuis son profil, après avoir confirmé son mot de passe. Sans commande, le compte est supprimé. Avec des commandes passées, il est **anonymisé** (nom remplacé, email fictif, compte désactivé) pour garder la comptabilité. La suppression est refusée tant qu'une commande est en cours.
- Le client peut modifier ses informations personnelles à tout moment.
- Les mentions légales et les CGV sont accessibles depuis le pied de page de toutes les pages.

Améliorations possibles

- Limiter le nombre de tentatives de connexion (composant RateLimiter de Symfony).
- Donner une durée de vie au jeton d'API, ou le stocker dans un cookie `HttpOnly` avec un front et une API sur le même domaine.
- Ajouter des en-têtes de sécurité HTTP (Content-Security-Policy) sur le front.

8. API REST

Toutes les routes sont documentées et testables sur `/api/doc`. Principales routes :

Ressource	Routes	Accès
Comptes	`POST /api/registration`, `/login`, `/verify-email`, `/resend-verification`, `/forgot-password`, `/reset-password`	Public
Mon compte	`GET /api/account/me`, `PUT /api/account/edit`, `DELETE /api/account`	Connecté
Menus, thèmes, régimes, allergènes	`GET` (liste et détail)	Public
	`POST`, `PUT`, `DELETE`	Employé
Plats	`GET`, `POST`, `PUT`, `DELETE /api/plat`	Employé (lecture : connecté)
Horaires	`GET /api/horraires` (public), `POST`, `PUT`, `DELETE`	Employé
Commandes (client)	`POST /api/commande/devis`, `POST /api/commande`, `GET /api/commande/me`, `PUT /api/commande/{id}`, `POST /api/commande/{id}/annuler`	Connecté (ses commandes)
Commandes (équipe)	`GET /api/commande?statut=&client=`, `PUT /api/commande/{id}/statut`	Employé
Avis	`GET /api/avis/publies` (public), `POST /api/avis`, `GET /api/avis/me`	Connecté
	`GET /api/avis`, `PUT /api/avis/{id}` (publier, refuser)	Employé
Utilisateurs	`POST /api/user/create-employe`, `GET /api/user`, `PUT` et `DELETE /api/user/{id}`	Administrateur
Statistiques	`GET /api/stats/commandes-par-menu`, `GET /api/stats/chiffre-affaires?menuId=&debut=&fin=`	Administrateur
Contact	`POST /api/contact`	Public

9. Accessibilité (RGAA)

Les pages ont été contrôlées avec l'outil **axe-core** (règles WCAG 2.1 niveaux A et AA, que reprend le RGAA). Le contrôle a porté sur les versions bureau et mobile, sur chaque onglet de l'espace client et du tableau de bord, et sur les fenêtres de détail et de commande. Résultat final : **aucune erreur détectée**.

Mesures en place :

- **Langue et titres de page** : `lang="fr"` sur la page, et un titre unique et explicite par page (« Nos menus - Vite & Gourmand »…).
- **Structure** : zones `header`, `main` et `footer`, un titre `h1` par page, une hiérarchie de titres sans saut de niveau.
- **Navigation au clavier** : lien « Aller au contenu » en début de page. Après chaque changement de page, le focus passe sur le nouveau contenu, ce qui guide les lecteurs d'écran.
- **Formulaires** : chaque champ a une étiquette reliée (`label for`). Les messages d'erreur et de succès sont annoncés (`role="alert"`, `role="status"`).
- **Boutons à icône** (afficher le mot de passe, menu mobile) : nommés avec `aria-label`. Les fenêtres modales sont reliées à leur titre (`aria-labelledby`).
- **Contrastes** : au moins 4,5:1 pour le texte. Deux nuances ont été ajoutées à la palette pour le texte : un corail foncé sur fond clair et un corail clair sur fond pétrole.
- **Images** : chaque image a un texte alternatif.

10. Tests

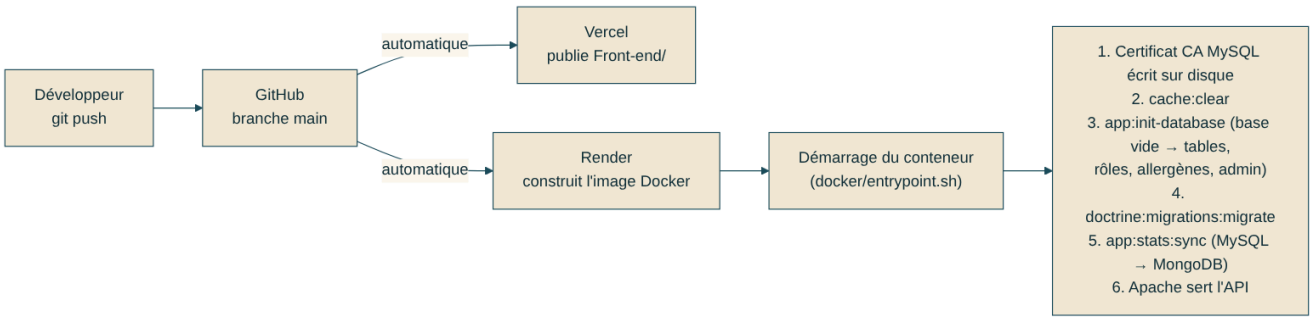
- **Tests automatisés** (PHPUnit, `tests/`) :
  - documentation de l'API accessible ;
  - routes protégées (401 sans jeton) ;
  - connexion avec un bon mot de passe, un mauvais mot de passe et un compte désactivé ;
  - statistiques interdites à un client (403) ;
  - règles de l'entité `User`.
- **Tests manuels** de chaque parcours sur le site en ligne : inscription et lien de confirmation, commande, modification, annulation, suivi des statuts, avis, gestion du catalogue, création d'employé, statistiques, formulaire de contact.
- **Contrôle du schéma** : `doctrine:schema:validate` après import des fichiers SQL.

11. Déploiement

Démarche

1. **Choisir des hébergeurs gratuits** pour chaque brique (partie 2), en séparant le front statique de l'API.
2. **Préparer l'API pour la production** :
   - un `Dockerfile` (PHP 8.3 + Apache + extensions `pdo_mysql`, `mongodb`, `intl`) ;
   - une configuration Apache qui envoie toutes les requêtes vers `public/index.php` ;
   - un script de démarrage `docker/entrypoint.sh` ;
   - la commande `app:init-database`, qui prépare une base vide sans intervention manuelle.
3. **Créer les services** dans cet ordre : bases de données et mails (Aiven, Atlas, Brevo), puis l'API (Render), puis le front (Vercel). Chaque étape a besoin des adresses de la précédente.
4. **Relier les services** par les variables d'environnement de Render. Les secrets n'apparaissent jamais dans le code.
5. **Vérifier en ligne** chaque parcours, puis corriger ce qui ne marchait qu'en local : les mails (SMTP bloqué → API de Brevo), le fuseau horaire (UTC → Europe/Paris), les chemins sensibles à la casse sous Linux (`/Assets/` → `/assets/`).

Mise à jour continue



Chaque push sur la branche main redéploie automatiquement le front et l'API ; les migrations de la base s'appliquent au démarrage.

Étapes détaillées

MySQL (Aiven) :

1. Créer un service MySQL en offre gratuite.
2. Récupérer l'hôte, le port, l'utilisateur, le mot de passe et le certificat CA.
3. Construire `DATABASE_URL` sous la forme `mysql://avnadmin:MOT_DE_PASSE@HOTE:PORT/defaultdb?serverVersion=8.0&charset=utf8mb4`.

MongoDB (Atlas) :

1. Créer un cluster M0.
2. Créer un utilisateur de base de données.

3. Autoriser l'accès réseau `0.0.0.0/0` (Render n'a pas d'adresse IP fixe).
4. Copier l'adresse `mongodb+srv://...` dans `MONGODB_URL`.

Brevo :

1. Valider l'adresse expéditrice.
2. Créer une clé API (`xkeysib-...`).
3. Renseigner `MAILER_DSN=brevo+api://CLE@default`.

Render :

1. Créer un « Web Service » depuis le dépôt GitHub, branche `main`, environnement Docker, offre gratuite.
2. Renseigner les variables : `APP_ENV=prod`, `APP_SECRET`, `DATABASE_URL`, `MYSQL_CA_CERT`, `MONGODB_URL`, `MONGODB_DB`, `MAILER_DSN`, `MAILER_FROM`, `CONTACT_EMAIL`, `FRONT_URL`, `CORS_ALLOW_ORIGIN`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`.
3. Au premier démarrage, l'API crée les tables, les rôles, les 14 allergènes et le compte administrateur.

Vercel :

1. Importer le dépôt avec `Front-end` comme dossier racine, sans commande de build.
2. Le fichier `vercel.json` renvoie toutes les adresses (`/menus`, `/profil`…) vers `index.html` pour le routeur.
3. Reporter ensuite l'adresse du front dans `FRONT_URL` et `CORS_ALLOW_ORIGIN` sur Render.

Le guide pas à pas complet se trouve dans le fichier `DEPLOIEMENT.md` du dépôt.

> **Limite de l'offre gratuite** : l'API Render se met en veille après 15 minutes sans visite et met environ une minute à se réveiller. Il faut donc ouvrir le site une minute avant une démonstration.